Time-consuming manualprocessesfor drafting complexsupportanswers.
Lack of unified database permission layers protecting private documentation from public adjustments.

Solution

The AI FAQ Assistant backend delivers a centralized, secure API wrapper that decouples application business logic from frontend delivery systems. It introduces internal verification matrices controlling specialized services for content creation, category keywords, database persistence, and automated text inference modules.

U sag e

Authenticated Creator Usage: Registers profiles, requests instant AI assistance, drafts official FAQ schemas, and updates or removes self-authored records.
Public/Consumer Usage: Queries general list routes to consume published articlesand issueslocalized semanticqueriesvia parameters.

Outcome

By offloading workflow controls to a robust backend, operational delays are minimized. Data structural integrity is strictly maintained through automated schema validation, security tokens guarantee that only original authors modify documents, and high-performance indexing facilitates real-time data discoverability.

1. Software Requirements

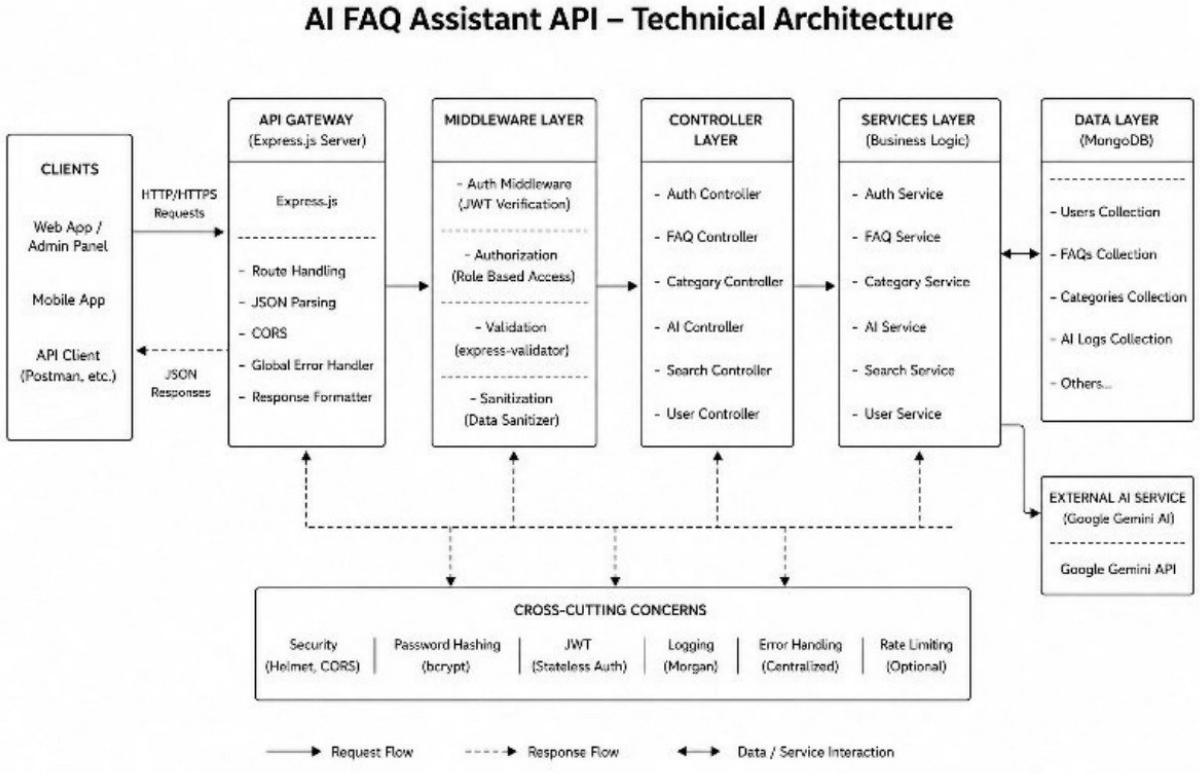
Operating System: Windows 10/11, macOS, or Linux (supports cross-platform operations).
Node.js (v18 or above): Runtime ecosystem running server logic and routing infrastructure.
npm (v9 or above): Node package manager to control operational dependencies.

Express.js (v4/v5 Ecosystem): Lightweight routing web framework to constructbackend RESTfulentrypoints.
MongoDB / Mongoose ODM: Document NoSQL engine storing application users and curated data entities.
Postman / ThunderClient: API verification toolkit to assert schema validation responses across administrative route shields.
Code Editor: Visual Studio Code or similar IDE environment.

2. Hardware Requirements

Processor: Intel Core i5 (8th Gen or above) / AMD Ryzen5orequivalent.
RAM: 8 GB minimum (16 GB recommended for concurrent instances of MongoDB processing layers).
Storage: 1 GB of available disk workspace.

Epic-1: Project Architecture



ComponentsDefinition

Interface: API tools like Postman query distinct route paths to extract JSONentities.
Express Server Gateway: Monitors network ports, maps explicit HTTP methods (GET, POST, PUT, DELETE), parses incoming JSON payloads, and handlesCORS headers.
Authentication Middleware: Extracts tokens from headers, validates user context signatures, and evaluates authorization thresholds for guarded resource paths.
Granular Services (Logic Modules): Enforces isolated domain rules for user session life cycles and operational query modifications.
Database Interface Layer: Mongoose translates operational application logic into performant native MongoDB queries.
AI Flow (Service Integration): Connected to external AI models via Google Gemini SDK integrations, letting the controller layer offload complex question-answering tasks and automated topic generation workflows.

Entity-Relationship (ER) DiagramDescription

2. FAQ Entity

_id(PrimaryKey - ObjectId)
question (String, Required)
answer (String, Required)
category (String, Required)
createdBy (Foreign Key referencing User Collection)
createdAt
updatedAt

3. AI Generation Entity (Logical)

(Generated dynamically using Google Gemini AI before persistence.)

topic (String)
generatedQuestion (String)
generatedAnswer (String)
generatedCategory (String)
generatedAt

4. Category Entity (Logical)

categoryName (String)
description (String)
totalFAQs (Number)

Key Structural Mappings

User → FAQ(One-to-Many):

A registered user can create and manage multiple FAQ records.

Category → FAQ (One-to-Many):
Each category can contain multiple FAQ documents.

User → AI Generated FAQ (One-to-Many):
A user can generate multiple AI-assisted FAQs, which may later be saved into the FAQ collection.

FAQ → Search Results (Logical Relationship):
Search queries retrieve matching FAQ documents using keyword and semantic matching.

Key Features

JWT Authentication & Authorization: Secure token-based authentication protects private API endpoints.
AI FAQ Generation: Google Gemini AI automatically generates questions, answers, and topic suggestions.
Semantic FAQ Search: Intelligent keyword-based search enables faster information retrieval.
Complete CRUD Operations:Create,Read,Update,andDeleteFAQrecords.
Category Management: Organizes FAQs into structured categories for easier navigation.
Password Encryption: User passwords are securely hashed using bcrypt before storage.
Centralized Error Handling: Standardized API error responses improve reliability and debugging.
Schema Validation: Mongoose validation ensures data integrity before databasepersistence.
RESTful API Architecture: Modular Express.js routes provide scalable backend services.
MongoDB Integration: Efficient NoSQL document storage using

Mongoose ODM.

Roles and Responsibilities

A dmi n

Manage all registered users.
View, update, and delete any FAQ.
Monitor AI-generated content.
Manage FAQ categories.
Access complete system functionality.

Content Creator

Create new FAQs.
Edit personal FAQ records.
DeleteownedFAQs.
Generate FAQs using AI.
Organize FAQs under categories.

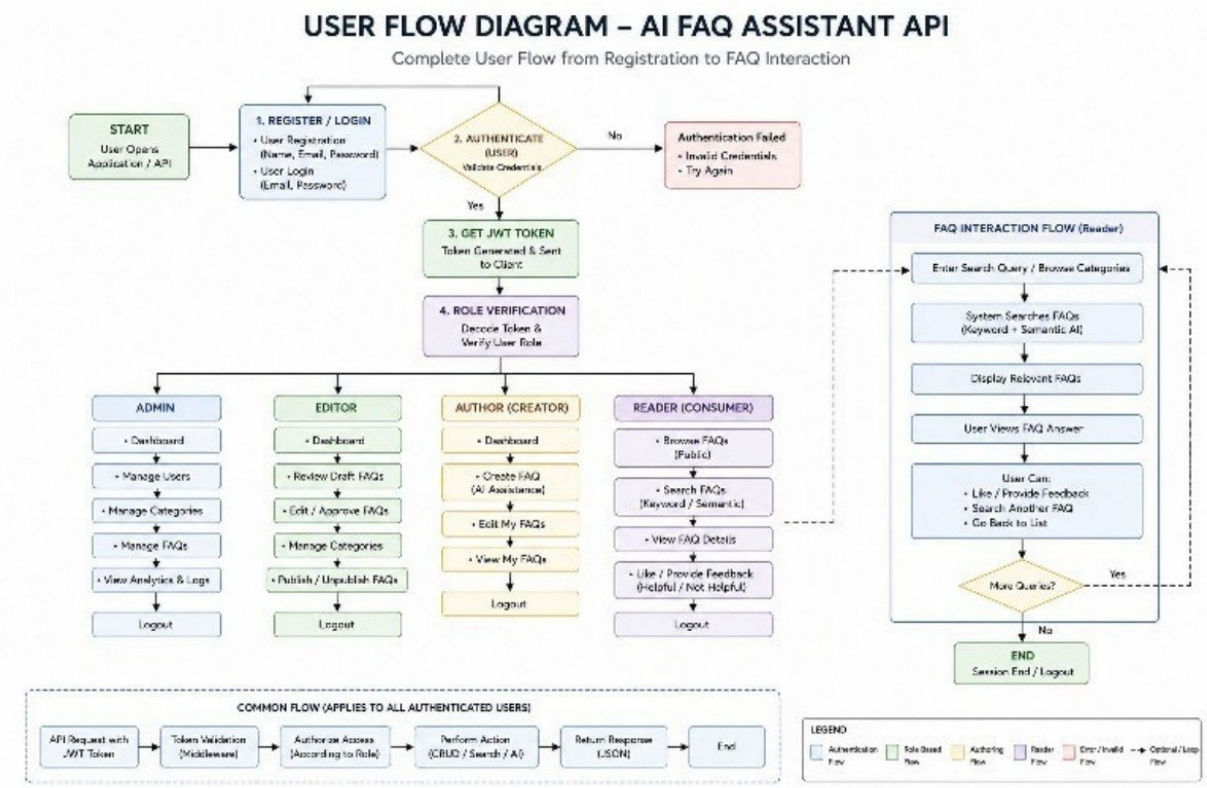
Authenticated User

View public FAQs.
Search FAQs using keywords.
Generate AI-assisted answers.
Access personal profile information.

Public User

View published FAQs.
Search FAQs.
Cannot create, edit, or delete FAQ records.
No access to protected API endpoints.

User Flow



MVC Architecture Pattern

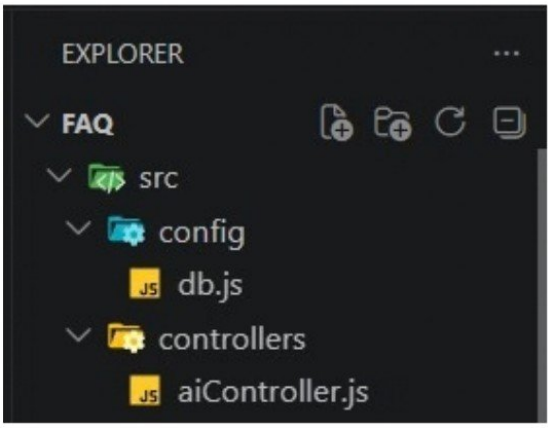
Thesystemarchitecture structures its source configuration layers around the Model-View-Controller design pattern to completely decouple components:

links network requests on specified paths directly to their associated controller handler logicmaps.

Epic2: Project Setup and Configuration

Creating Project Folder

1. Createa new folder with your <projectname>.
2. Inside that folder, create a new subfolder named src.
3. Now openthatfolder inVS Code.



Server Setup (npm init)

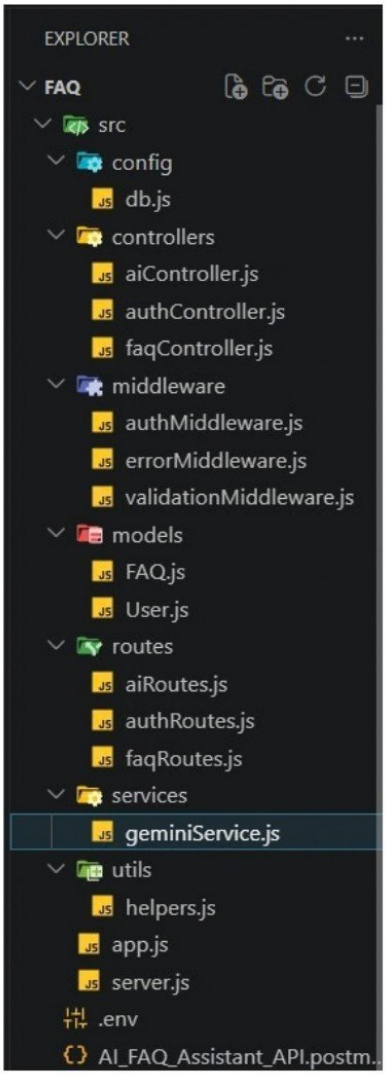
Open your root folder terminal in VS Code and run:

Bash

```
npm init-y
npm install express mongoose dotenv cors jsonwebtoken bcrypt @google/genai
```

File and Folder Structure Matrix

Plaintext

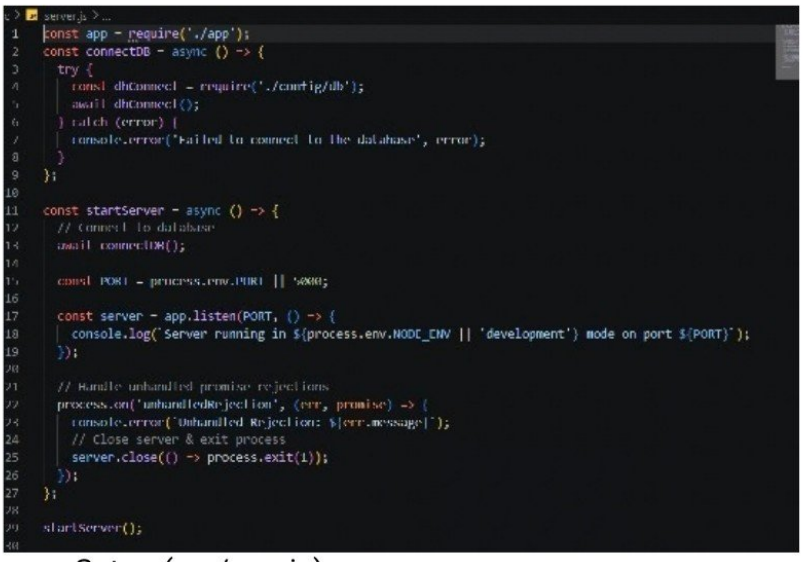


Epic3: Backend Development

Core SystemScript Implementations

1. Gateway Server Node (src/server.js)

This main entrance mechanism aggregates environment variables, triggers databaselisteners, and bindsnetwork listenerstoactiveenvironmentports:



```
const app = require('./app');
const connectDB = async () => {
  try {
    const dbConnect = require('./config/db');
    await dbConnect();
  } catch (error) {
    console.error('Failed to connect to the database', error);
  }
};

const startServer = async () => {
  // Connect to database
  await connectDB();

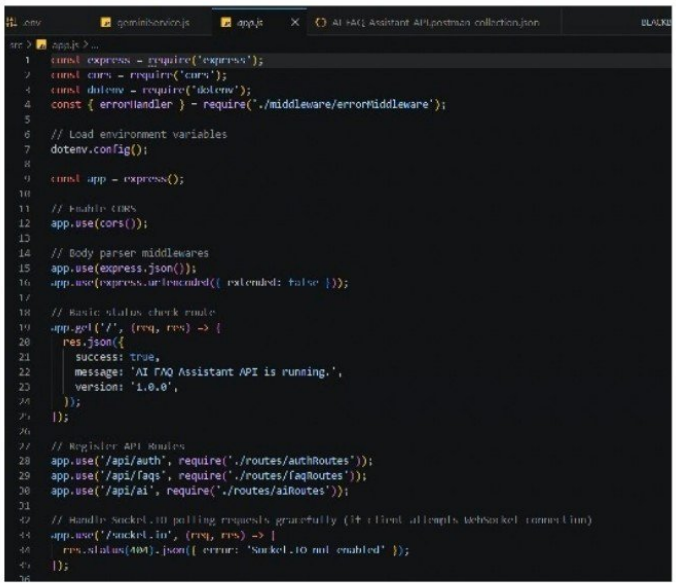
  const PORT = process.env.PORT || 5000;

  const server = app.listen(PORT, () => {
    console.log(`Server running in ${process.env.NODE_ENV || 'development'} mode on port ${PORT}`);
  });

  // Handle unhandled promise rejections
  process.on('unhandledRejection', (err, promise) => {
    console.error(`Unhandled Rejection: ${err.message}`);
    // Close server & exit process
    server.close(() => process.exit(1));
  });
};

startServer();
```

2. Gateway Express Setup (src/app.js)



```
const express = require('express');
const cors = require('cors');
const dotenv = require('dotenv');
const { errorHandler } = require('./middleware/errorMiddleware');

// Load environment variables
dotenv.config();

const app = express();

// Enable CORS
app.use(cors());

// Body parser middlewares
app.use(express.json());
app.use(express.urlencoded({ extended: false }));

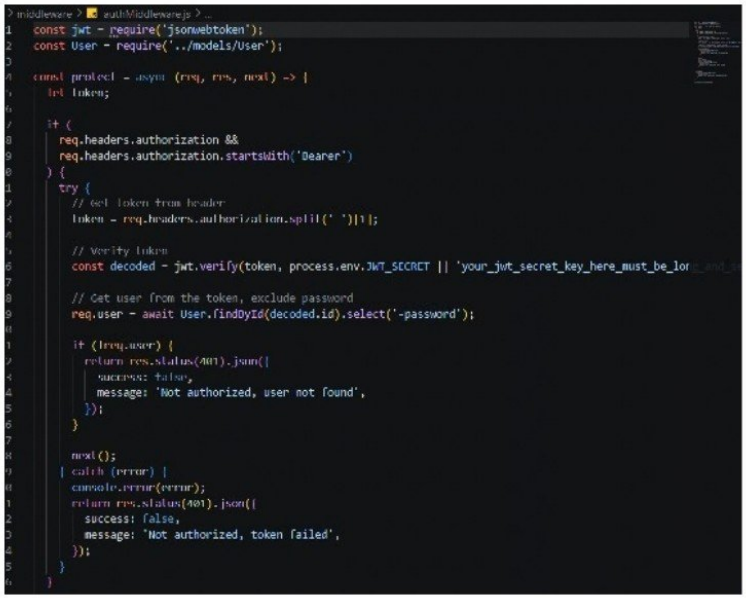
// Basic status check route
app.get('/', (req, res) => {
  res.json({
    success: true,
    message: 'AI FAQ Assistant API is running.',
    version: '1.0.0',
  });
});

// Register API Routes
app.use('/api/auth', require('./routes/authRoutes'));
app.use('/api/faqs', require('./routes/faqRoutes'));
app.use('/api/ai', require('./routes/aiRoutes'));

// Handle Socket.IO polling requests gracefully (if client attempts WebSocket connection)
app.use('/socket.io', (req, res) => {
  res.status(404).json({ error: 'Socket.IO not enabled' });
});
```

3. Security Access Middleware (src/middleware/authMiddleware.js)

Intercepts incoming target queries, extracts transmission header authorization records, and validates token identity context claims:



```
const jwt = require('jsonwebtoken');
const User = require('../models/User');

const protect = async (req, res, next) => {
  let token;

  if (
    req.headers.authorization &&
    req.headers.authorization.startsWith('Bearer')
  ) {
    try {
      // Get token from header
      token = req.headers.authorization.split(' ')[1];

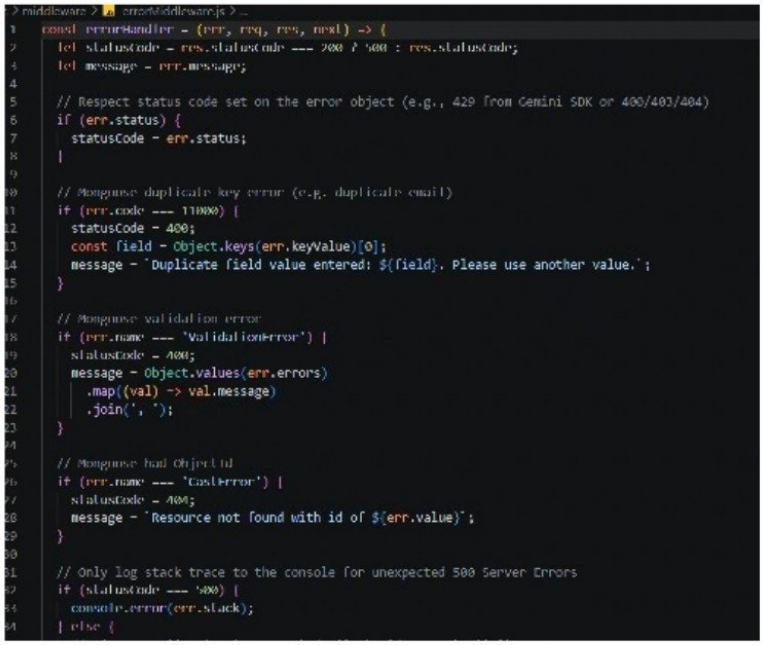
      // Verify token
      const decoded = jwt.verify(token, process.env.JWT_SECRET || 'your_jwt_secret_key_here_must_be_lor

      // Get user from the token, exclude password
      req.user = await User.findById(decoded.id).select('-password');

      if (!req.user) {
        return res.status(401).json({
          success: false,
          message: 'Not authorized, user not found',
        });
      }

      next();
    } catch (error) {
      console.error(error);
      return res.status(401).json({
        success: false,
        message: 'Not authorized, token failed',
      });
    }
  }
```

4. Centralized Error Handler Middleware (src/middleware/errorMiddleware.js)



```
const errorHandler = (err, req, res, next) => {
  let statusCode = res.statusCode === 200 ? 500 : res.statusCode;
  let message = err.message;

  // Respect status code set on the error object (e.g., 429 from Gemini SDK or 400/403/404)
  if (err.status) {
    statusCode = err.status;
  }

  // Mongoose duplicate key error (e.g. duplicate email)
  if (err.code === 11000) {
    statusCode = 400;
    const field = Object.keys(err.keyValue)[0];
    message = `Duplicate field value entered: ${field}. Please use another value.`;
  }

  // Mongoose validation error
  if (err.name === 'ValidationError') {
    statusCode = 400;
    message = Object.values(err.errors)
      .map((val) => val.message)
      .join(', ');
  }

  // Mongoose bad ObjectId
  if (err.name === 'CastError') {
    statusCode = 404;
    message = `Resource not found with id of ${err.value}`;
  }

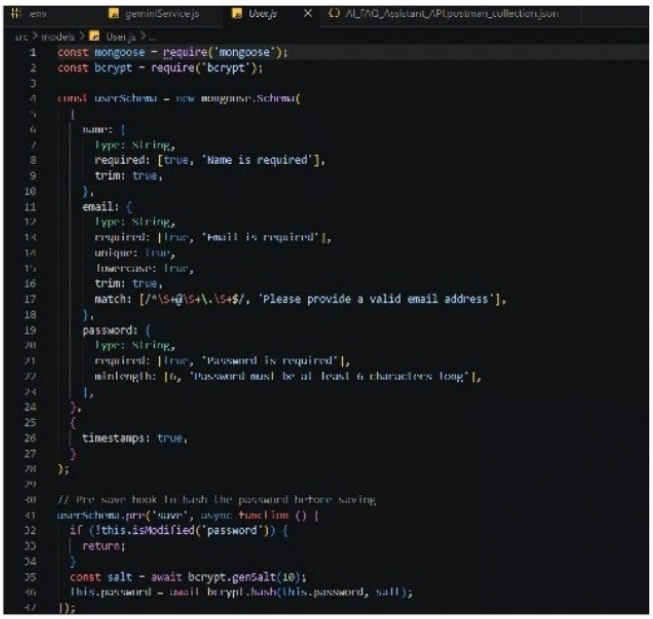
  // Only log stack trace to the console for unexpected 500 Server Errors
  if (statusCode === 500) {
    console.error(err.stack);
  } else {
```

Epic4: Database Configuration

Mongoose SchemaDefinitions

1. Identity Resource Document Schema (src/models/User.js)

Enforces profile field criteria validations and hooks password hashing steps prior to user document persistence:



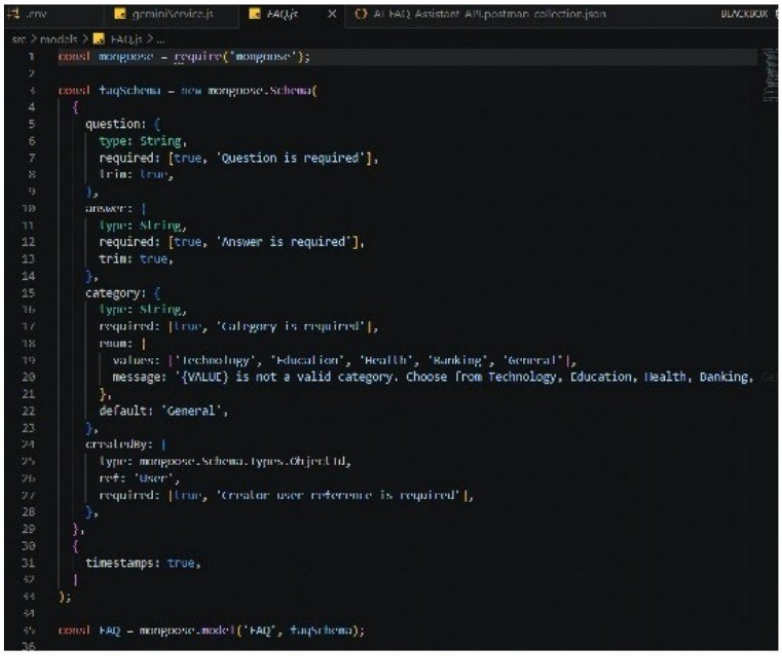
```
const mongoose = require('mongoose');
const bcrypt = require('bcrypt');

const userSchema = new mongoose.Schema(
  {
    name: {
      type: String,
      required: [true, 'Name is required'],
      trim: true,
    },
    email: {
      type: String,
      required: [true, 'Email is required'],
      unique: true,
      lowercase: true,
      trim: true,
      match: [/^\S+@\S+\.\S+$/, 'Please provide a valid email address'],
    },
    password: {
      type: String,
      required: [true, 'Password is required'],
      minlength: [6, 'Password must be at least 6 characters long'],
    },
  },
  {
    timestamps: true,
  }
);

// Pre save hook to hash the password before saving
userSchema.pre('save', async function () {
  if (!this.isModified('password')) {
    return;
  }
  const salt = await bcrypt.genSalt(10);
  this.password = await bcrypt.hash(this.password, salt);
});
```

2. FAQ Resource Document Schema (src/models/FAQ.js)

Models core properties and maps a distinct relational reference link back to the source creator document:



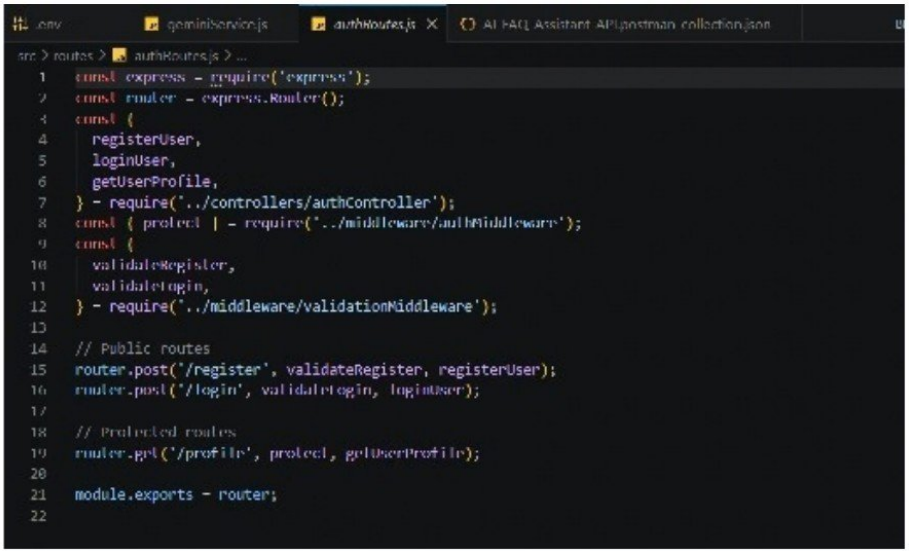
```
const mongoose = require('mongoose');

const faqSchema = new mongoose.Schema(
  {
    question: {
      type: String,
      required: [true, 'Question is required'],
      trim: true,
    },
    answer: {
      type: String,
      required: [true, 'Answer is required'],
      trim: true,
    },
    category: {
      type: String,
      required: [true, 'Category is required'],
      enum: {
        values: ['Technology', 'Education', 'Health', 'Banking', 'General'],
        message: '{VALUE} is not a valid category. Choose from Technology, Education, Health, Banking,
      },
      default: 'General',
    },
    createdBy: {
      type: mongoose.Schema.Types.ObjectId,
      ref: 'User',
      required: [true, 'Creator user reference is required'],
    },
  },
  {
    timestamps: true,
  }
);

const FAQ = mongoose.model('FAQ', faqSchema);
```

API Router Interface Endpoints

1. Public Authentication Operations (src/routes/authRoutes.js)



```
const express = require('express');
const router = express.Router();
const {
  registerUser,
  loginUser,
  getUserProfile,
} = require('../controllers/authController');
const { protect } = require('../middleware/authMiddleware');
const {
  validateRegister,
  validateLogin,
} = require('../middleware/validationMiddleware');

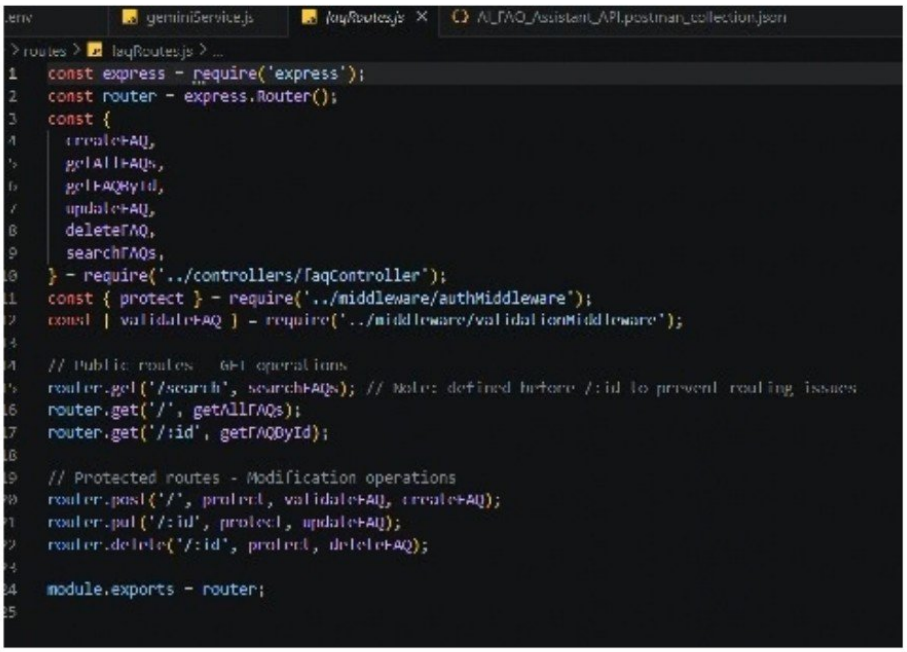
// Public routes
router.post('/register', validateRegister, registerUser);
router.post('/login', validateLogin, loginUser);

// Protected routes
router.get('/profile', protect, getUserProfile);

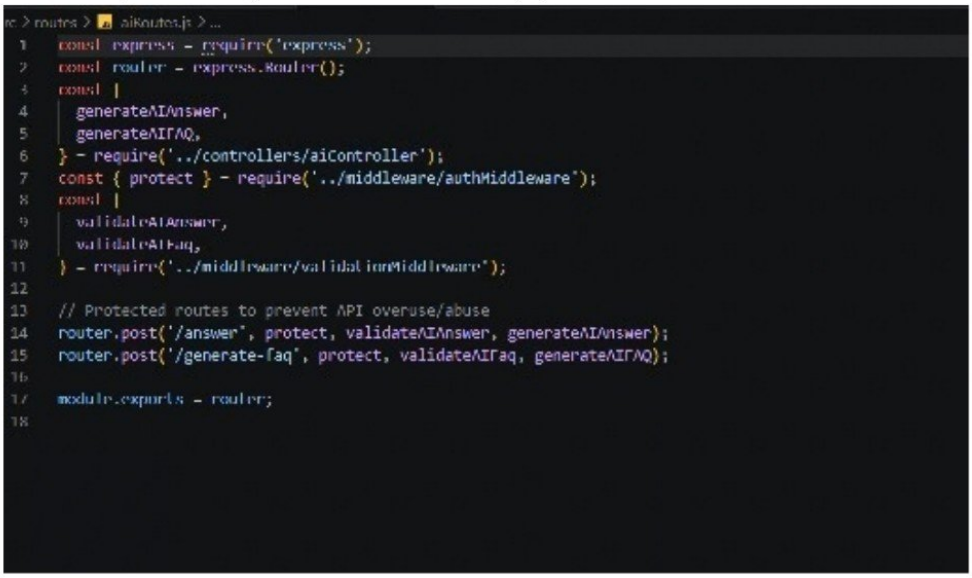
module.exports = router;
```

2.ProtectedFAQ Operations (src/routes/faqRoutes.js)

JavaScript



3. AI Automation Operations (src/routes/aiRoutes.js)



Epic5: Project Executions

Running the API Application

Install Postman or ThunderClient(VSCodeextension).

Start the backend development server environment parameters:

Bash

```
npm run dev
```

API Testing Specification Demo

To verify operational structural integrity via execution endpoints, map endpoints using the configurations below:

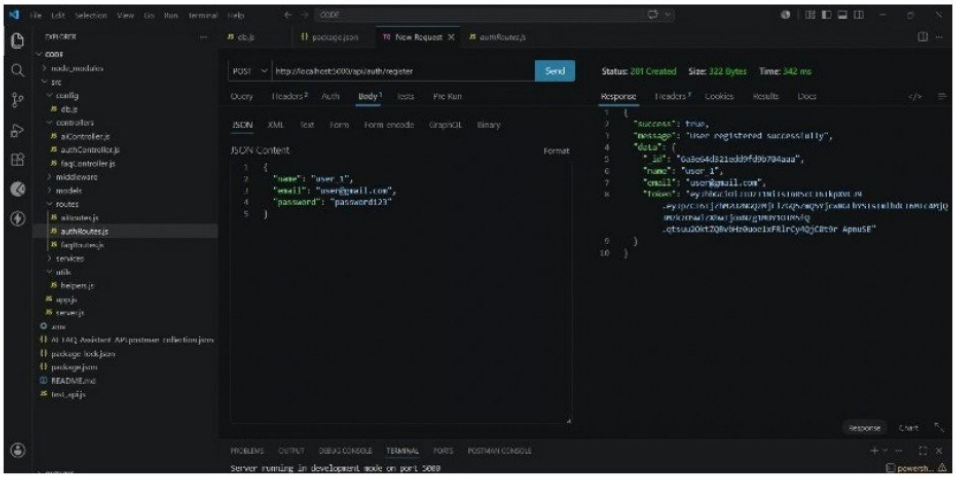
1. Identity Onboarding Path Registration

Route: POST /api/auth/register
Access:Public
Payload Request Body JSON:

JSON

```
{
 "name": "Priya Sharma",
 "email": "priya@writeflow.com",
"password": "securepassword123"
}
```

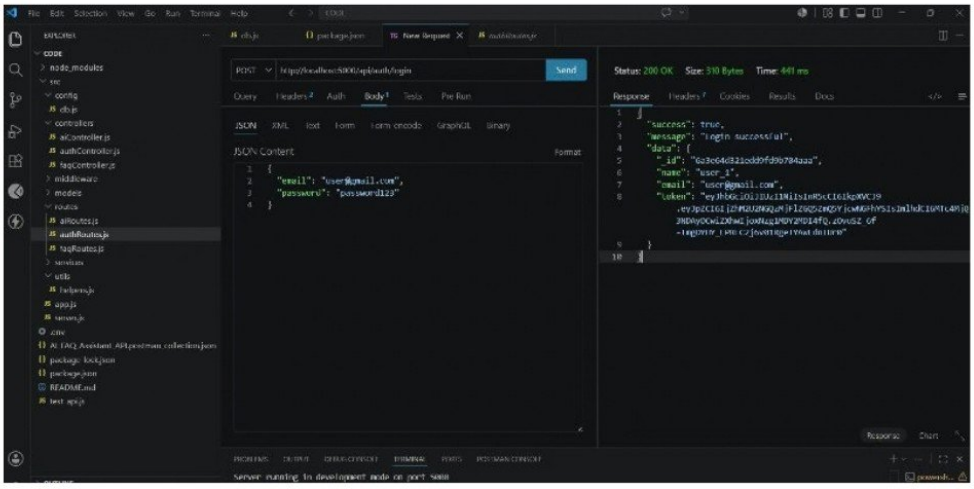
2. Profile Login Verification Quey



Route: POST /api/auth/login
Access:Public
Payload Request Body JSON:

JSON

```
 {
 "email": "priya@writeflow.com",
"password": "securepassword123"
}
```

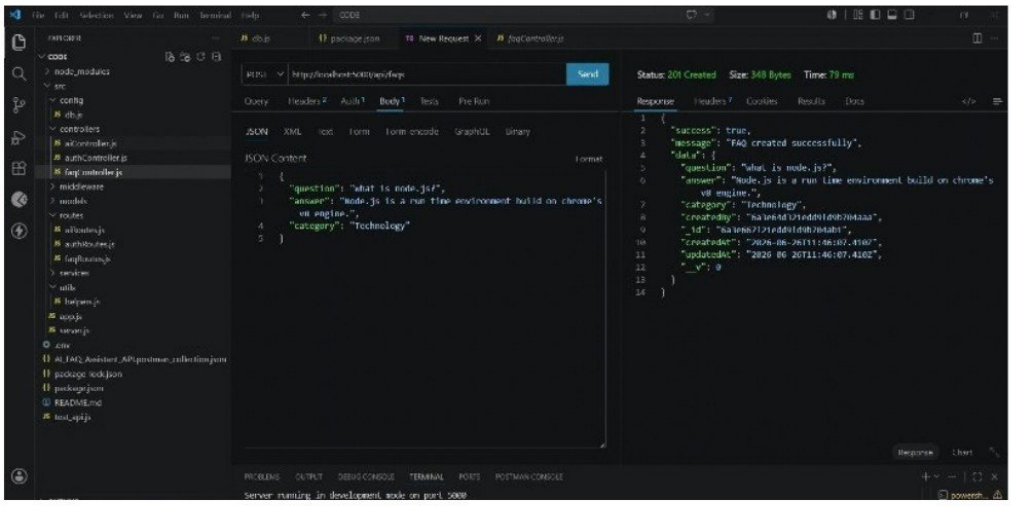


3. Resource Entity Creation (FAQ)

Route: POST /api/faqs
Access: Private (Bearer <JWT Token>)
Payload Request Body JSON:

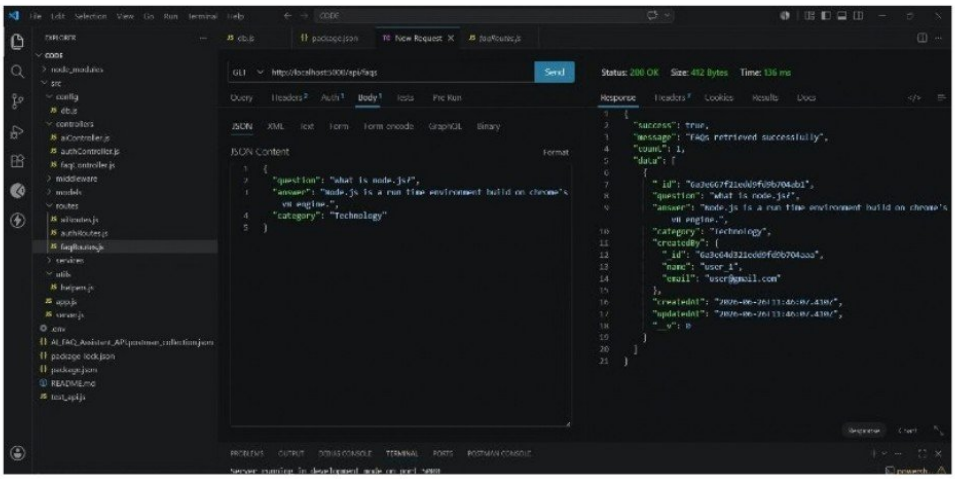
JSON

```
 {
 "question": "How do I configure custom category tags?",
"answer": "Navigate to settings panel, append keyword       tags arrays, and
save schema.",
 "category": "Configuration"
}
```



4. KeywordResource Entity Fetching (Search)

Route: GET/api/faqs/search?q=configure
Access:Public



5. AI Intelligent Content Automation (Generate FAQ Pair)

Route: POST/api/ai/generate-faq
Access: Private (Bearer <JWT Token>)
Payload Request Body JSON:

JSON

```
 {
"topic": "Mongooseschemaindexingvalidationruntimeworkflowoptimization"
}
```

API Testing video:
https://drive.google.com/file/d/1iyv31Ce9ridpLFALod6PlCbkfHyrWmlN/view?usp=drivesdk

demo vido:Demo link
:https://drive.google.com/file/d/1Q51OlLvhEKzeQBqTGzn4zDrCKCi0zlji/view?usp=drivesdk